CS 543 • WEB SERVICES • GROUP 7

# AEGISAI

## Enterprise AI Security & Governance Gateway

Complete Assignment Submission Documentation

Assignments 1–6 • HTTP • REST • SOAP • Contracts • Resilience • Validation

### Submission Resources

| | |
|---|---|
| GitHub | github.com/neha-nupur/AegisAi |
| Google Drive Demo | Open Google Drive demonstration folder |

Sharing: Anyone with the link → Viewer

# Contents

AegisAI is presented as a continuous progression from HTTP fundamentals to service architecture, external integration, REST implementation, HTTP semantics, and robust validation.

**Contents** 2

**1. Project Overview** 4

Project goals . . . 5

**2. Problem and Goals** 6

**3. End-to-End AI Request Flow** 7

**4. Main Services and Responsibilities** 9

**5. Assignment-by-Assignment Submission Guide** 10

Assignment 1 --- HTTP by Hand & Project Setup . . . 10

Assignment 2 --- Services, Contracts & Schema . . . 10

Assignment 3 --- Integrate an External SOAP Partner . . . 11

Assignment 4 --- Model the Service, Publish the Contract, Implement It, Survive the Network . . . 11

Assignment 5 --- HTTP Methods & Headers . . . 12

Assignment 6 --- Honest Responses, Errors & Validation . . . 13

Assignment progression at a glance . . . 13

**6. Model & Provider Service API** 15

Example: register a provider . . . 15

Model routing and invocation . . . 16

**7. HTTP Behaviour and Error Handling** 17

Idempotency, caching, and concurrency . . . 18

Security and privacy considerations . . . 18

**8. Technology and Protocols** 19

**9. Repository Structure** 20

**10. Running the Model & Provider Service** 21

Option A --- Python/Flask Assignment 4 . . . 21

Option B --- JavaScript Model & Provider service (Assignments 4--6) . . . 21

**11. Validation and Test Evidence** 22

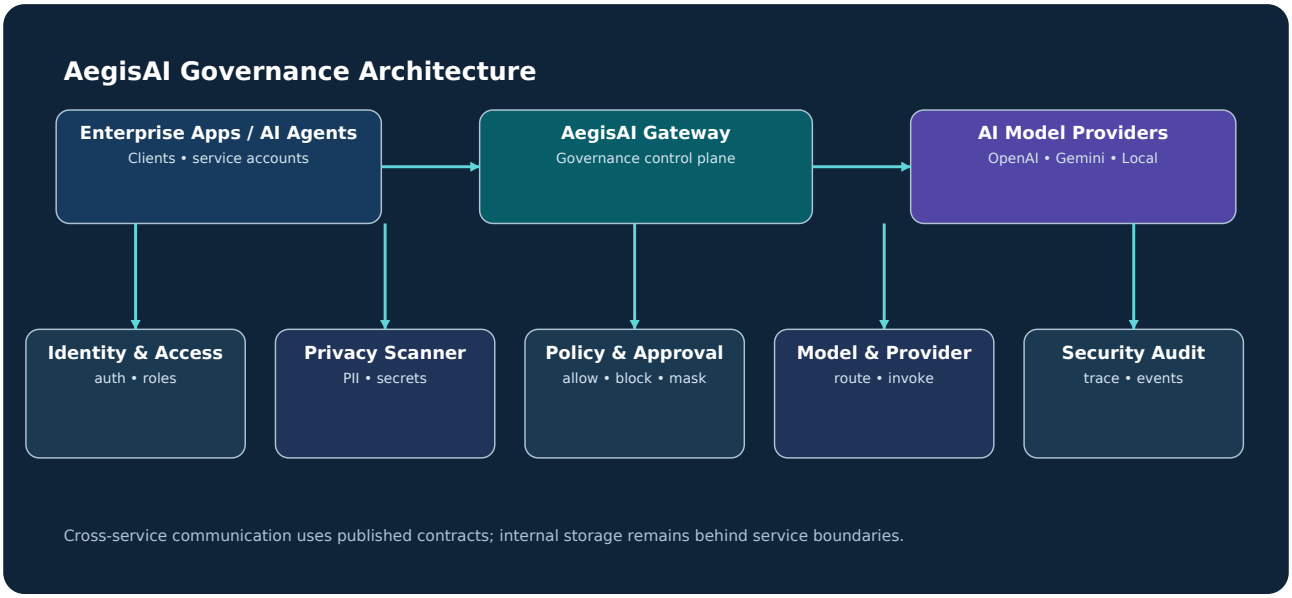
**12. Team** 23

**13. Learning Outcomes** 24

**Final Summary** 25

**AegisAI — System Synthesis** 26

# 1. Project Overview




Applications often integrate directly with different AI providers. If every application independently implements authentication, privacy checks, model selection, retries, and audit logs, the rules can become inconsistent and difficult to inspect.

AegisAI proposes a centralized control plane:

```
Enterprise Applications / AI Agents
                |
                v
        +------------------+
        |     AegisAI      |
        | Governance       |
        | Gateway          |
        +------------------+
                |
      +---------+----------+
      |         |          |
      v         v          v
 Identity    Privacy     Policy &
 & Access    Scanner     Approval
      |         |          |
      +---------+----------+
                |
                v
          Model & Provider
              Routing
                |
                v
       External AI Providers
       OpenAI | Gemini | Local
                |
                v
       Response Filtering
                |
                v
        Security Audit /
        Trace & Observability
```

The AI model is treated as an external dependency. AegisAI's purpose is to govern the request around that dependency, rather than merely forwarding prompts.

## Project goals

- Authenticate callers and apply role- or permission-based access controls.
- Validate requests before processing them.
- Detect or mask potentially sensitive information before it reaches a provider.
- Apply organization-defined policy decisions.
- Support approval workflows for requests that require human review.
- Select a suitable model based on capability, sensitivity, policy, cost, and provider health.
- Reduce duplicate processing through idempotency controls.
- Enforce request limits and provide predictable HTTP responses.
- Handle provider failures using timeouts, retries, and circuit-breaker behaviour.
- Preserve audit records and correlation identifiers for troubleshooting.
- Describe both modern REST interfaces and a legacy SOAP/WSDL partner boundary.

# 2. Problem and Goals

Without a shared gateway, separate applications may use different security rules, expose sensitive data accidentally, retry failed requests unsafely, or lack a complete record of why a model was selected.

AegisAI addresses these risks through a consistent request pipeline. It is designed around four primary actor types:

Actor Typical responsibilities

---------- ----------

Organization Administrator Manages organization settings, users, access, policies, models, providers, and limits.

Developer Integrates an application or agent, submits AI requests, and views permitted request history.

Security Auditor Reviews security events, policy outcomes, audit records, and traces.

AI Agent / Service Account Authenticates machine-to-machine requests and accesses only permitted capabilities.

The broader design includes Identity & Access, AI Gateway, Privacy Scanner, Policy & Approval, Model & Provider, and Security Audit responsibilities. Assignment folders focus on the deliverables specified for each stage rather than claiming that every architectural component is implemented as a single fully deployed product.

# 3. End-to-End AI Request Flow




A typical governed request follows this sequence:




Important design principle: a request should not be passed directly to an AI provider before the relevant validation, privacy, and policy checks have run. Failures should be represented honestly with appropriate HTTP status codes rather than disguising an error as a successful response.

# 4. Main Services and Responsibilities

The architecture separates responsibilities into services so each capability has a clear owner.

Service / component Responsibility

--- ---

Identity & Access Organizations, users, service accounts, authentication, and authorization.

AI Gateway Main entry point; coordinates validation, idempotency, limits, policy, routing, and the request lifecycle.

Privacy Scanner Detects configured sensitive-data patterns and supports masking/redaction decisions.

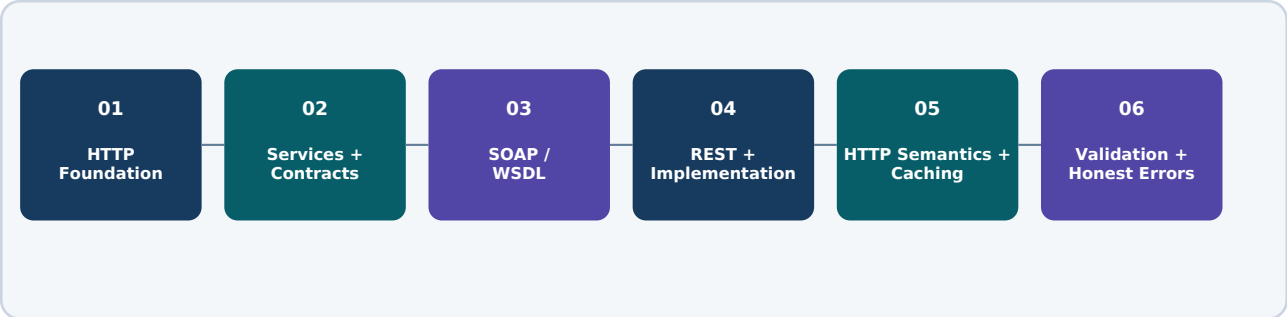
Policy & Approval Evaluates governance rules and supports `ALLOW`, `BLOCK`, `MASK`, `REQUIRE_APPROVAL`, and routing decisions.

Model & Provider Registers providers and models, exposes their capabilities, selects eligible models, invokes providers, and reports provider health.

Security Audit Stores or exposes security events, decisions, request traces, and evidence needed for investigation.

Supporting infrastructure in the wider design includes PostgreSQL for relational/business data, MongoDB for document or trace-oriented data, and Redis for rate-limit or short-lived coordination state. These are architectural roles; the exact datastore used depends on the assignment component being demonstrated.

# 5. Assignment-by-Assignment Submission Guide




The assignments form a learning progression from protocol observation to service design and implementation.

## Assignment 1 --- HTTP by Hand & Project Setup

Purpose: Establish the HTTP fundamentals and define AegisAI before designing its service contracts.

Deliverables: - `docs/http-log.md` --- annotated `curl -i` request/response examples, including successful responses and deliberate error/redirect cases. - `docs/network-analysis.md` --- analysis of a real page load, including requests, transferred data, and observed response behaviour. - `README.md` --- project overview and repository documentation. - `brief.md` --- system brief describing the purpose, actors, core nouns, and operation families.

Flow: Observe HTTP on the wire → understand status codes and headers → inspect real network behaviour → define the system and its vocabulary.

## Assignment 2 --- Services, Contracts & Schema

Purpose: Turn the AegisAI brief into a service-oriented design, with explicit capabilities, service ownership, contracts, and data structures.

Deliverables: - `Assignment 2/Task 1, 3, 4, 6/AegisAi.pdf` --- capability catalogue, service ownership, service contracts, specification of the central `submitAIRequest` operation, and service-property validation. - `Assignment 2/Task 2/AegisAIServices.drawio` --- editable service architecture diagram. - `Assignment 2/Task 2/AegisAIServices.drawio.png` --- rendered architecture diagram. - `Assignment 2/Task 5/AegisAiERDiagram.drawio` --- editable entity-relationship diagram. - `Assignment 2/Task 5/AegisAiERDiagram.drawio.png` --- rendered ER

diagram. - `Assignment 2/Task 5/AegisAiSchema.sql` --- SQL schema.

Flow: Identify capabilities → assign each capability to an owning service → define service contracts → specify the central AI-request operation → validate service properties → map the data model.

Design principle: Service ownership should be explicit. The architecture and schema are intended to make responsibilities and relationships understandable before implementation.

## Assignment 3 --- Integrate an External SOAP Partner

Purpose: Demonstrate how AegisAI can consume a legacy SOAP service while keeping its own service contracts separate from the partner's contract.

The scenario uses a hypothetical Meridian Messaging SMS notification partner. The `sendSMS` operation represents notifying an approver when an AI request is waiting for human approval.

Deliverables: - `Assignment 3/integration.pdf` --- integration context, HTTP binding, discovery/registry entry, and fault mapping. - `Assignment 3/partner.wsdl` --- WSDL 1.1 contract defining types, messages, port type, binding, service, and endpoint. - `Assignment 3/soap-request.xml` --- example SOAP request. - `Assignment 3/soap-response.xml` --- example SOAP success response. - `Assignment 3/soap-fault.xml` --- example SOAP fault. - `Assignment 3/verification.txt` --- automated structural, schema, signature, and fault checks.

Flow: Define the partner contract → construct a request envelope → map the WSDL to the HTTP/SOAP exchange → describe discovery → translate partner faults into AegisAI-level errors.

Boundary: This assignment is a design-time integration exercise. It does not claim that a live Meridian service or a running AegisAI SOAP endpoint is deployed.

## Assignment 4 --- Model the Service, Publish the Contract, Implement It, Survive the Network

Purpose: Apply REST resource modelling to the Model & Provider service and implement the contract with basic resilience.

The repository includes a Python/Flask implementation in `Assignment 4/` and a JavaScript implementation that the later assignments extend in `Assignment-4(Model-and-Provider)/`.

Python deliverables (`Assignment 4/`): - Part A --- service-model notes. - Part B --- `openapi.yaml` and contract-validation output. -

Part C --- Flask implementation, data models, store, error handling, and automated tests. - Part D --- provider client and provider stub for network-failure exercises. - `Evidence/curl-transcript.txt` --- request/response evidence. - `README.md`, `RUNME.md`, setup and run scripts.

JavaScript baseline (`Assignment-4(Model-and-Provider)/`): - `openapi.yaml` --- REST contract. - `app.js`, `routes.js`, `models.js`, `store.js`, `validators.js`, and `errors.js` --- service implementation modules. - `providerClient.js`, `routeInvoke.js`, and `routing.js` --- provider communication and model-selection logic. - `tests/service.test.js` --- automated tests. - `NOTES.md` --- design decisions and written answers.

Flow: Model nouns as resources → define URLs and HTTP methods → publish the OpenAPI contract → validate inputs → implement endpoints → invoke a provider through a controlled client → test success and failure cases.

## Assignment 5 --- HTTP Methods & Headers

Purpose: Extend the JavaScript Model & Provider service with correct HTTP method semantics, conditional requests, caching, discovery, and retry-safety behaviour.

Main folder: `Assignment5/`

Key deliverables: - Updated REST routes and resource operations. - `openapi.yaml` --- updated contract. - `routesExtra.js` and `routesProvider.js` --- additional route handling. - `validators.js`, `errors.js`, `providerClient.js`, `routeInvoke.js`, and related service modules. - `curl-transcript.txt` --- HTTP exchange evidence. - `NOTES.md` --- method, header, caching, and idempotency rationale. - `tests/service.test.js` --- automated tests.

Topics demonstrated: - `GET`, `POST`, `PATCH`, `DELETE`, and `OPTIONS`. - `Location` on resource creation. - `ETag`, `If-None-Match`, and `304 Not Modified`. - `If-Match` and `412 Precondition Failed` for stale updates. - `Idempotency-Key` for retry-safe create requests. - Query parameters for filtering, sorting, and pagination. - `Allow`, CORS preflight, and documented method-override behaviour. - Correct distinction between safe and idempotent operations.

Flow: Establish resource semantics → apply the appropriate method → return meaningful headers → make caching and concurrency explicit → document and test the wire-level behaviour.

## Assignment 6 --- Honest Responses, Errors & Validation

Purpose: Harden the service contract so that validation failures, missing resources, conflicts, authentication issues, rate limits, and provider failures are represented consistently.

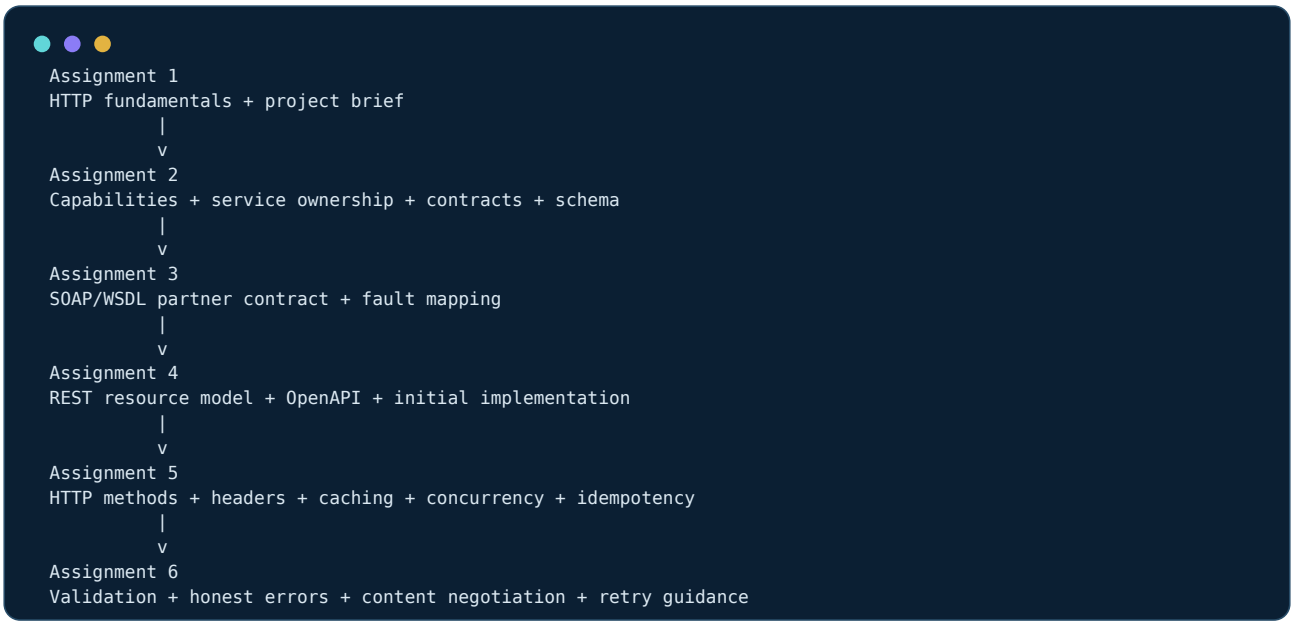
Main folder: `Assignment6/`

Key deliverables: - `app.js` and route modules --- endpoint and middleware behaviour. - `validators.js` --- request-field validation. - `errors.js` --- consistent error responses. - `openapi.yaml` --- published contract. - `transcript.txt` --- captured request/response evidence. - `NOTES.md` --- status-code map, error catalogue, retry guidance, and JSON/XML negotiation notes. - `tests/service.test.js` --- automated tests.

Topics demonstrated: - Clear distinction between malformed input, invalid fields, missing resources, and state conflicts. - Stable Problem Details-style error bodies with machine-readable `type`, `title`, `status`, and `detail`. - Field-level validation errors. - Correct status codes instead of returning `200 OK` for failed operations. - Safe exposure of errors: internal stack traces, provider details, and sensitive implementation information should not be sent to clients. - `Retry-After` for retryable conditions such as rate limiting or temporary provider unavailability. - Content negotiation for JSON/XML where supported by the implementation. - Retry guidance that distinguishes transient failures from requests that must be corrected first.

Flow: Validate the request → classify the failure → return the correct HTTP status and error body → avoid leaking internal details → document retry semantics → verify behaviour with captured exchanges and tests.

## Assignment progression at a glance


Assignment 1
HTTP fundamentals + project brief
|
v
Assignment 2
Capabilities + service ownership + contracts + schema
|
v
Assignment 3
SOAP/WSDL partner contract + fault mapping
|
v
Assignment 4
REST resource model + OpenAPI + initial implementation
|
v
Assignment 5
HTTP methods + headers + caching + concurrency + idempotency
|
v
Assignment 6
Validation + honest errors + content negotiation + retry guidance

# 6. Model & Provider Service API

The Model & Provider service manages provider definitions and the models available through them. It also selects a suitable model and invokes the provider for a request.

The JavaScript implementation uses the `/v1` API prefix.

Method Endpoint Purpose

---------- ---------- ----------

`POST /v1/providers` Register a provider.

`GET /v1/providers/{providerId}` Retrieve a provider.

`PATCH /v1/providers/{providerId}` Partially update a provider, subject to the implementation's precondition rules.

`DELETE /v1/providers/{providerId}` Remove a provider and its associated models where applicable.

`POST /v1/providers/{providerId}/models` Register a model under a provider.

`GET /v1/models` List models, with filtering, sorting, and pagination options.

`GET /v1/providers/{providerId}/models/{modelId}` Retrieve a model.

`POST /v1/route-and-invoke` Select a suitable model and invoke its provider.

`GET /v1/providers/{providerId}/health` Read provider health and circuit state.

`OPTIONS /v1/providers`, `/v1/models` Discover supported methods.

The published `openapi.yaml` files are the source of truth for the detailed request and response schemas for each assignment version. Check the OpenAPI file in the relevant folder because the API evolves between assignments.

## Example: register a provider

```
POST /v1/providers HTTP/1.1
Content-Type: application/json
Accept: application/json
Authorization: Bearer <demo-token>
Idempotency-Key: provider-registration-001
```

```
{
  "name": "OpenAI",
  "type": "openai"
}
```

A successful creation should return a creation status (typically `201 Created`) and a `Location` header identifying the newly created provider. The exact fields and accepted values are defined in the relevant OpenAPI contract.

## Model routing and invocation

`POST /v1/route-and-invoke` is a processing operation rather than ordinary CRUD. It selects an eligible model at invocation time, applies the relevant request/policy constraints, and calls the provider through a client that can enforce timeouts, retries, and circuit-breaker behaviour. If a provider cannot return a valid result, the service should return an explicit failure rather than inventing a model completion.

# 7. HTTP Behaviour and Error Handling

Assignments 4--6 progressively demonstrate how a web service should communicate its behaviour through HTTP.

Status Typical meaning in this project

---------- ----------

`200 OK` A read, update, health response, or successful processing operation completed.

`201 Created` A provider or model resource was created; include `Location` where appropriate.

`204 No Content` An operation succeeded without a response body, such as a successful deletion or an `OPTIONS` response.

`304 Not Modified` The client's cached representation is still current.

`400 Bad Request` The request cannot be parsed or its query parameters are malformed.

`401 Unauthorized` Authentication is missing or invalid.

`404 Not Found` The requested resource or route does not exist.

`406 Not Acceptable` The server cannot provide a representation matching the client's `Accept` header.

`409 Conflict` The request conflicts with existing resource state, such as a duplicate provider name.

`412 Precondition Failed` A conditional update failed because its `If-Match` precondition was stale.

`422 Unprocessable Content` The request body is syntactically valid but fails field or semantic validation.

`429 Too Many Requests` The caller's request budget or rate limit has been exceeded.

`503 Service Unavailable` No suitable model is available, a provider is unavailable, or a circuit is open.

`504 Gateway Timeout` A provider did not respond within the configured timeout.

The exact status used for a particular validation case follows the assignment version's contract. The key rule is that a failed request must not be disguised as a successful `200 OK`.

## Idempotency, caching, and concurrency

- `Idempotency-Key` can make repeated resource-creation requests safe when the same key and request are replayed.
- `ETag` and `If-None-Match` allow clients to avoid downloading unchanged representations.
- `If-Match` prevents a client from silently overwriting a newer representation.
- `Retry-After` communicates when a client should retry a retryable rate-limit or temporary-service failure.
- `GET` and `OPTIONS` are safe; idempotency and retry-safety for state-changing requests depend on the method and implementation contract.

## Security and privacy considerations

- Use environment variables for credentials and API keys; do not commit real secrets.
- Keep error responses free of stack traces, local file paths, credentials, and internal provider details.
- Validate request fields before using them in routing or persistence logic.
- Treat privacy scanning and policy enforcement as controls that reduce risk, not as a guarantee of perfect detection.

# 8. Technology and Protocols

The repository contains more than one implementation language because the assignments build the service in stages.

| Area | Technologies / formats used in the assignment work |
| --- | --- |
| REST service implementation | JavaScript / Node.js and Express-style modules; Python / Flask in the separate Assignment 4 implementation. |
| API contracts | OpenAPI YAML. |
| SOAP integration design | WSDL 1.1, SOAP 1.1, XML, XSD, SOAP Faults, and WS-Security examples. |
| Data modelling | SQL schema and entity-relationship diagram. |
| Test evidence | Automated service tests, contract validation, verification output, and `curl` transcripts. |
| Wider AegisAI architecture | PostgreSQL, MongoDB, Redis, external AI providers, audit/trace storage, and containerized deployment concepts. |
| Diagrams and documentation | Draw.io, Markdown, PDF, XML, SQL, and YAML. |

The presence of a technology in the wider architecture does not mean every assignment folder independently configures or deploys it.

# 9. Repository Structure

```
AegisAi/
├── README.md
├── ReadMe.pdf
├── brief.md
├── docs/
│   ├── http-log.md
│   ├── network-analysis.md
│   └── ...
├── Assignment 2/
│   ├── Task 1, 3, 4, 6/
│   │   └── AegisAi.pdf
│   ├── Task 2/
│   │   ├── AegisAIServices.drawio
│   │   └── AegisAIServices.drawio.png
│   └── Task 5/
│       ├── AegisAiSchema.sql
│       ├── AegisAiERDiagram.drawio
│       └── AegisAiERDiagram.drawio.png
├── Assignment 3/
│   ├── integration.pdf
│   ├── partner.wsdl
│   ├── soap-request.xml
│   ├── soap-response.xml
│   ├── soap-fault.xml
│   └── verification.txt
├── Assignment 4/
│   ├── README.md
│   ├── RUNME.md
│   ├── requirements.txt
│   ├── Evidence/
│   ├── Part A - Model the Service/
│   ├── Part B - Publish the Contract/
│   ├── Part C - Implement It/
│   └── Part D - Survive the Network/
├── Assignment-4(Model-and-Provider)/
│   ├── openapi.yaml
│   ├── app.js
│   ├── routes.js
│   ├── models.js
│   ├── store.js
│   ├── validators.js
│   ├── errors.js
│   ├── providerClient.js
│   ├── routeInvoke.js
│   ├── routing.js
│   ├── tests/
│   └── NOTES.md
├── Assignment5/
│   ├── openapi.yaml
│   ├── app.js
│   ├── routes*.js
│   ├── validators.js
│   ├── errors.js
│   ├── tests/
│   ├── curl-transcript.txt
│   └── NOTES.md
└── Assignment6/
    ├── openapi.yaml
    ├── app.js
    ├── routes*.js
    ├── validators.js
    ├── errors.js
    ├── tests/
    ├── transcript.txt
    └── NOTES.md
```

This is a guide to the main submission files; consult the actual folder contents for the complete file list.

# 10. Running the Model & Provider Service

## Option A --- Python/Flask Assignment 4

The Python implementation includes Windows helper scripts.

1. Open the `Assignment 4/` folder.
2. Run `setup.bat` once to create the virtual environment and install dependencies.
3. Start the provider stub by running `run_provider_stub.bat`.
4. Start the service in a second terminal using `run_service.bat`.
5. Run `run_tests.bat` to execute the automated tests.
6. See `RUNME.md` for the full steps and expected output.

The supplied run instructions use port `6001` for the provider stub and port `6000` for the service. Both windows should remain open while manually testing the interaction.

Example request:

```
curl -i -X POST http://localhost:6000/providers \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"OpenAI\",\"base_url\":\"http://localhost:6001\"}"
```

## Option B --- JavaScript Model & Provider service (Assignments 4--6)

Each JavaScript assignment folder is a self-contained iteration. Run commands from the folder you want to test, not from the repository root.

1. Install the dependencies listed in that folder's `package.json`.
2. Review that folder's `NOTES.md` and `openapi.yaml`.
3. Start the service using the script defined in that folder's `package.json`.
4. Run the tests using the folder's test script.
5. Use the corresponding `curl-transcript.txt` or `transcript.txt` to compare the expected HTTP behaviour.

For example:

```
cd "Assignment6"
npm install
npm test
```

If the package's scripts differ, use the values in that folder's `package.json`. Provider invocation may require a provider stub or configuration; follow the notes for the specific assignment version.

# 11. Validation and Test Evidence

Evidence is kept alongside the relevant deliverables so each assignment can be reviewed independently.

- Assignment 3: `verification.txt` records 27 passed, 0 failed checks for WSDL structure, XML schema conformance, request element order, WS-Security signature/digest checks, and SOAP fault structure.
- Assignment 4 (Python): `Part C - Implement It/pytest-output.txt` records 8 passed tests, including creation, idempotent replay, malformed bodies, unknown IDs, conflicts, filtered model lists, and provider health behaviour.
- Assignments 4--6 (JavaScript): test suites and captured HTTP transcripts are included in their respective folders. Review each folder's test output and transcript for the evidence corresponding to that iteration.
- Assignment 4 (OpenAPI): `Part B - Publish the Contract/openapi-validation.txt` records the contract validation result.

These are the recorded results included with the submitted files; they are not a claim that every folder has been executed in the current environment.

# 12. Team

AegisAI is a Group 7 submission for CS 543 --- Web Services.

Role Team member Roll number

---------- ------------------- ---------------

Group Leader Aditi Garg 20251651008

Group Member Neha Nupur 20251651064

Group Member Shivam Kumar Soni 20251651084

# 13. Learning Outcomes

Across the assignments, the submission demonstrates the progression from protocol understanding to service design and implementation:

- Reading HTTP requests, responses, headers, and status codes.
- Translating a system brief into capabilities and service boundaries.
- Assigning data and operations to service owners.
- Designing an ER diagram and SQL schema.
- Describing a SOAP/WSDL contract and mapping partner faults.
- Modelling REST resources and publishing an OpenAPI contract.
- Implementing and validating request handlers.
- Applying HTTP method semantics, conditional requests, caching, and idempotency.
- Returning consistent errors and field-level validation details.
- Considering timeouts, retries, circuit breakers, and safe error exposure.
- Maintaining test outputs and HTTP transcripts as reviewable evidence.

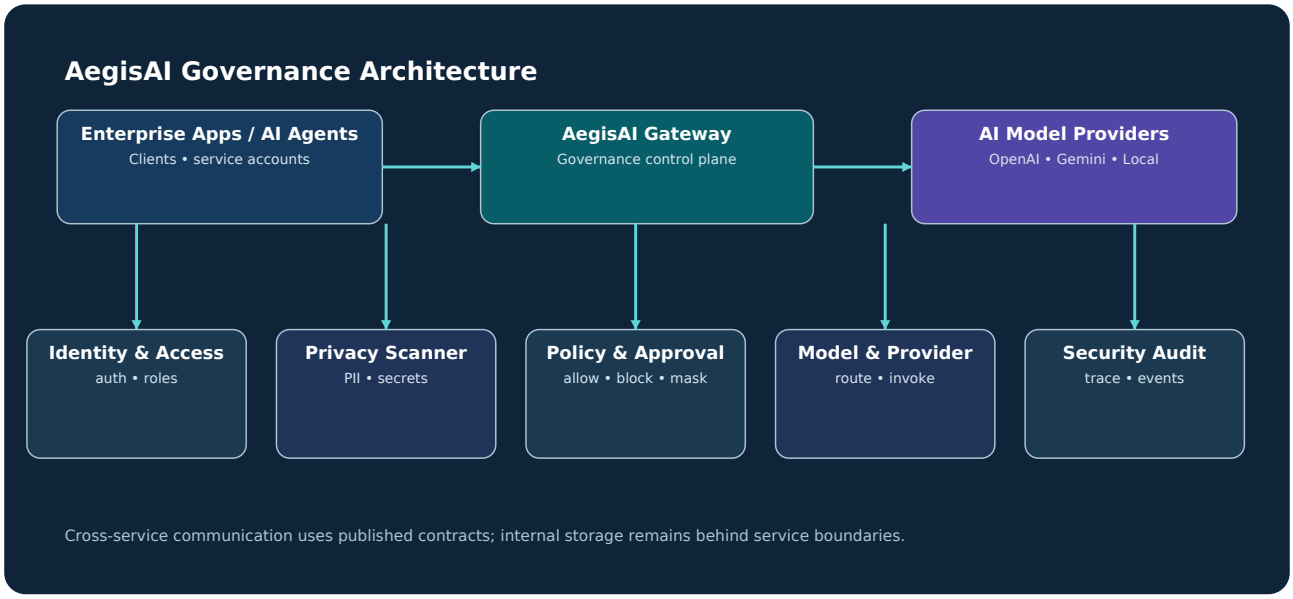
# Final Summary

**AegisAI is designed as a governed pathway between enterprise AI consumers and model providers.** Assignments 1--3 establish the HTTP foundation, service architecture, data model, and external SOAP contract. Assignments 4--6 then focus on a concrete Model & Provider REST service, progressing through resource modelling and implementation to HTTP correctness, validation, and reliable error handling.

For the detailed contract of any endpoint, always consult the `openapi.yaml` in the specific assignment folder. For design rationale and evidence, consult the corresponding `NOTES.md`, PDF, test output, and HTTP transcript.

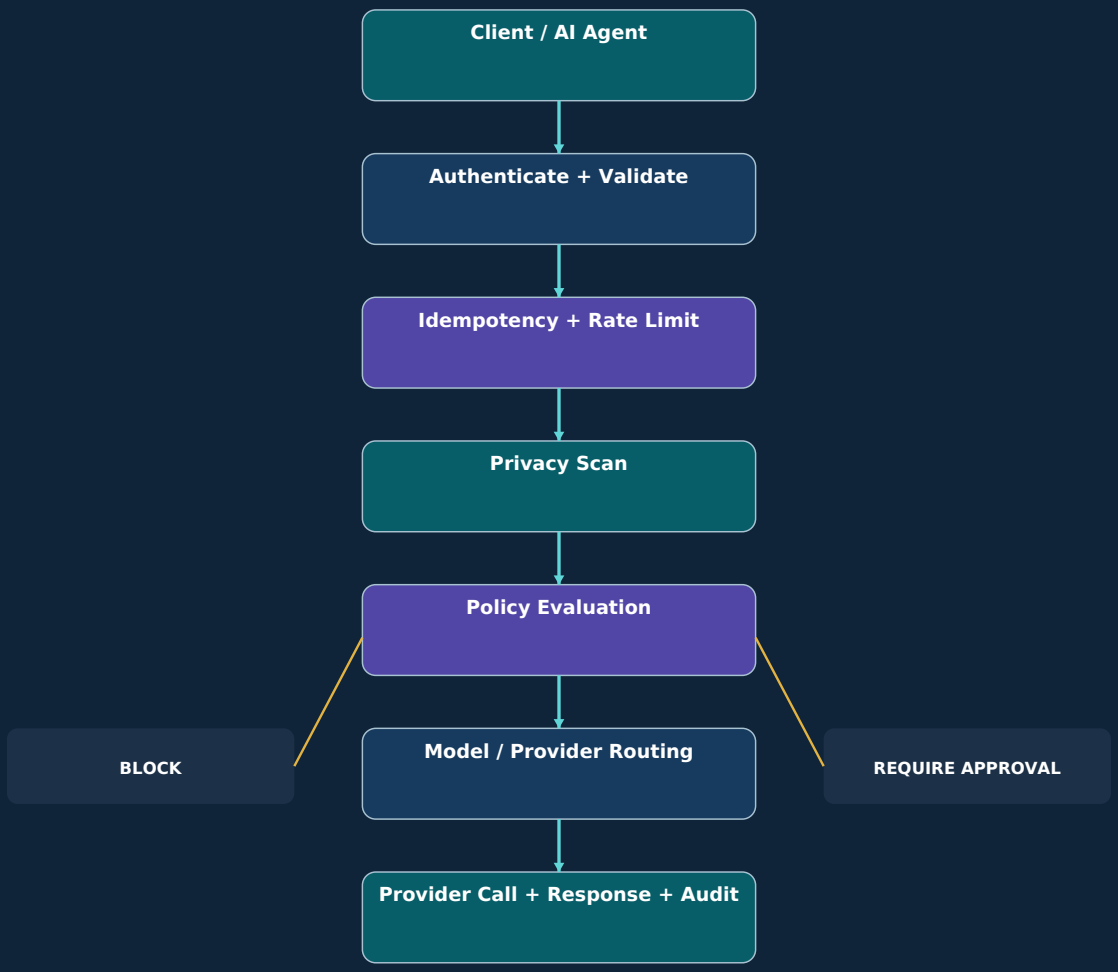
# AegisAI — System Synthesis

The architecture below brings together the service boundaries and the governed request path. It provides a compact technical map of the complete submission.

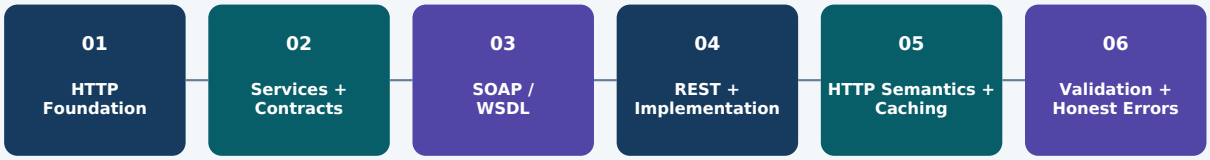


## Governed AI Request Lifecycle

Security, privacy, policy, routing, resilience and audit controls




Decision branches stop, pause, transform, or route a request before provider execution.




## Submission Links

**GitHub Repository** https://github.com/neha-nupur/AegisAi

**Google Drive Demonstration** Open Google Drive demonstration folder

Sharing: Anyone with the link → Viewer